

INTRODUCCIÓN A LA PROGRAMACIÓN ORIENTADA A OBJETOS

Genericidad

Dr. Luciano H. Tamargo
http://cs.uns.edu.ar/~lt
Depto. de Ciencias e Ingeniería de la Computación
Universidad Nacional del Sur, Bahía Blanca

CASO DE ESTUDIO: FILA GENÉRICA.

```
FilaRobot
f: arreglo de Robot
cant: entero

<<Constructores>>
FilaRobot(n: entero)
<<Comandos>>
insertar(r: Robot)
<<Consultas>>
cantRobots(): entero
esCreciente(): boolean
filaLlena(): boolean
contar(r: Robot): entero
robotsMenosEnergia(e: entero): entero
```

Se comporta como una colección

insertar(r: Robot)
Requiere r ligada y cant menor a la cantidad de componentes del arreglo.

contar(r: Robot)
Requiere r ligada. Cuenta la cantidad de robots equivalentes a r.

La clase **Fila** encapsula a un arreglo **f** cuyas componentes son de tipo **Robot**.

CASO DE ESTUDIO: FILA GENÉRICA.

```
FilaRacionales
f: arreglo de Racional
cant: entero

<<Constructores>>
FilaRacionales(n: entero)
<<Comandos>>
insertar(r: Racional)
<<Consultas>>
cantRacionales(): entero
esCreciente(): boolean
filaLlena(): boolean
contar(r: Racional): entero
cantCeros(): entero
```

Se comporta como una colección

insertar(r: Racional)
Requiere r ligada y cant menor a la cantidad de componentes del arreglo.

contar(r: Racional)
Requiere r ligada. Cuenta la cantidad de racionales equivalentes a r.

La clase **Fila** encapsula a un arreglo **f** cuyas componentes son de tipo **Racional**.

GENERICIDAD

- Una **clase genérica** encapsula a una estructura cuyo comportamiento es independiente del tipo de las componentes.

- La genericidad aumenta las oportunidades de reuso, porque evita escribir el mismo código repetidamente.
- Las clases genéricas pueden especializarse en otras que exhiben un comportamiento más ligado a la aplicación y son menos reusables.

CASO DE ESTUDIO: FILA GENÉRICA.

```
FilaGenerica
f: arreglo de Elemento
cant: entero

<<Constructores>>
FilaGenerica(n: entero)
<<Comandos>>
insertar(e: Elemento)
<<Consultas>>
cantElementos(): entero
esCreciente(): boolean
filaLlena(): boolean
contarElementos(e: Elemento): entero
```

Se comporta como una colección

insertar(e: Elemento)
Requiere e ligada y cant menor a la cantidad de componentes del arreglo.

contarElementos(e: Elemento)
Requiere e ligada. Cuenta la cantidad de elementos equivalentes a e.

La clase **Fila** encapsula a un arreglo **f** cuyas componentes son de tipo **Elemento**.

CASO DE ESTUDIO: FILA GENÉRICA.

```
class FilaGenerica{
    //Atributos de instancia
    protected Elemento [] f;
    protected int cantElementos;
    public FilaGenerica(int n){
        f = new Elemento[n];
        cantElementos = 0;
    }
    public void insertar(Elemento e){
        f[cantElementos] = e; cantElementos++;
    }
    public int cantElementos(){
        return cantElementos;
    }
}
```

CASO DE ESTUDIO: FILA GENÉRICA.

```
class FilaGenerica{
...
public boolean esCreciente(){
    //Requiere cantElementos() >= 2
    int i = 0;
    boolean es = true;
    while (i< cantElementos()-1 && es){
        es = f[i].menor(f[i+1]);
        i++;
    }
    return es;
}
...
}
```

- El método `esCreciente()` retorna verdadero sí y solo si los elementos almacenados en una fila `f` están ordenados de forma creciente.
- Se asume que `f` tiene al menos dos elementos.

CASO DE ESTUDIO: FILA GENÉRICA.

```
class FilaGenerica{
...
public boolean esCreciente(){
    //Requiere cantElementos() >= 2
    int i = 0;
    boolean es = true;
    while (i< cantElementos()-1 && es){
        es = f[i].menor(f[i+1]);
        i++;
    }
    return es;
}
...
}
```

- El código del método `esCreciente` es independiente del tipo de las componentes del arreglo `f`, en tanto esté definida la relación `menor`.

CASO DE ESTUDIO: FILA GENÉRICA.

```
...
public boolean filaLlena(){
    return cant == fila.length;
}

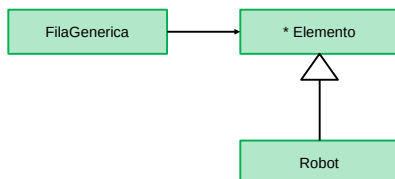
public int contarElementos(Elemento e){
    int i = 0;
    int cont = 0;
    while (i < cantElementos()){
        if (f[i].equals(e))
            cont++;
        i++;
    }
    return cont;
}
}
```

CASO DE ESTUDIO: FILA GENÉRICA.

```
abstract class Elemento {
    abstract boolean equals(Elemento e);
    abstract boolean menor(Elemento e);
}
```



CASO DE ESTUDIO: FILA GENÉRICA.



CASO DE ESTUDIO: FILA GENÉRICA.

- En la aplicación se considera que:
 - Un robot es igual a otro si tiene el mismo nombre. El método `equals` compara los atributos nombre.
 - Un robot es menor que otro si tiene menos energía. El método `menor` compara los atributo energía.

CASO DE ESTUDIO: FILA GENÉRICA.

```
class Robot extends Elemento{
    ...
    public boolean equals(Elemento e){
        //Requiere e ligado
        Robot r = (Robot) e;
        return nombre.equals(r.obtenerNombre());
    }
    public boolean menor(Elemento e){
        //Requiere e ligado
        Robot r = (Robot) e;
        return energia < (r.obtenerEnergia());
    }
}
```

¿Por qué es necesario el casting?

CASO DE ESTUDIO: FILA GENÉRICA.

```
class testFabrica{
    ...
    FilaGenerica f = new FilaGenerica(100);
    Robot rob1 = new Robot("Bob");
    Robot rob2 = new Robot("Tom");
    if(!f.filaLlena() && rob1!=null) f.insertar(rob1);
    if(!f.filaLlena() && rob2!=null) f.insertar(rob2);
    ...
    if (f.esCreciente())
    ...
}
```

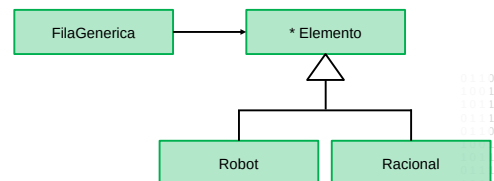
- Aunque una fila contiene componentes de tipo Elemento, en una aplicación particular todas las componentes tienen instancias del mismo tipo, en este ejemplo Robot.

CASO DE ESTUDIO: FILA GENÉRICA.

```
class testFabrica{
    ...
    FilaGenerica f = new FilaGenerica(100);
    Robot rob1 = new Robot("Bob");
    Robot rob2 = new RobotAlfa("Tom");
    if(!f.filaLlena() && rob1!=null) f.insertar(rob1);
    if(!f.filaLlena() && rob2!=null) f.insertar(rob2);
    ...
    if (f.esCreciente())
    ...
}
```

Si la clase Robot es especializada por otra clase RobotAlfa, las componentes de una fila pueden ser instancias de Robot o de RobotAlfa.
La estructura en este caso es genérica y polimórfica.

CASO DE ESTUDIO: FILA GENÉRICA.



CASO DE ESTUDIO: FILA GENÉRICA.

```
class Racional extends Elemento{
    //num y den no tienen factores primos
    ...
    public boolean equals(elemento e){
        //Requiere e ligado
        Racional r = (Racional) e;
        return r.num() == num && r.den() == den;
    }
    public boolean menor(elemento e){
        //Requiere e ligado
        Racional r = (Racional) e;
        return num/den < r.num()/r.den();
    }
}
```

CASO DE ESTUDIO: FILA GENÉRICA.

```
class testRacionales{
    ...
    FilaGenerica f = new FilaGenerica(100);
    Racional r1 = new Racional(1,2);
    Racional r2 = new Racional(1,3);
    if(!f.filaLlena() && r1!=null) f.insertar(r1);
    if(!f.filaLlena() && r2!=null) f.insertar(r2);
    ...
    if (f.esCreciente())
    ...
}
```

- Aunque un objeto de clase FilaGenerica tiene componentes de tipo Elemento, en una aplicación particular todas las componentes son instancias del mismo tipo, en este ejemplo Racional

CASO DE ESTUDIO: FILA GENÉRICA.

- Un objeto de clase **FilaGenerica** puede contener elementos de clase **Robot** o de clase **Racional**, pero no debería contener algunos de clase **Racional** y otros de clase **Robot**.
- Si el programador de clase cliente de **FilaGenerica**, por error usara insertar algunas veces con parámetros de clase **Racional** y otras con parámetros de clase **Robot**, el método **esCreciente** no funcionará correctamente.
- No está definido un método **menor** que compare un **Robot** con un objeto de clase **Racional** y probablemente no tenga sentido hacerlo.

19

CASO DE ESTUDIO: FILA GENÉRICA.

```
class testFila{
...
FilaGenerica f = new FilaGenerica(2);
Racional r1 = new Racional (1,2);
Racional r2 = new Racional (1,3);
Racional r3 = new Racional (1,2);
f.insertar(r1);
f.insertar(r2);
f.insertar(r3);
}
```

Error de ejecución, en este caso la clase cliente no cumple el contrato.

20

CASO DE ESTUDIO: FILA GENÉRICA.

```
class testFila{
...
FilaGenerica f = new FilaGenerica(3);
Racional r1 = new Racional (1,2);
Racional r2 = null;
Racional r3 = new Racional (1,2);
f.insertar(r1);
f.insertar(r2);
f.insertar(r3);
if (f.esCreciente())
}
```

Error de ejecución, en este caso la clase cliente no cumple el contrato.

21

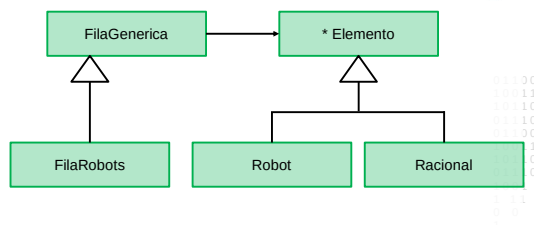
CASO DE ESTUDIO: FILA GENÉRICA.

```
class testFila{
...
FilaGenerica f = new FilaGenerica(10);
Racional r1 = new Racional (1,2);
Racional r2 = new Racional (1,3);
Robot r3 = new Robot("Tom");
f.insertar(r1);
f.insertar(r2);
f.insertar(r3);
...
}
```

Error de la aplicación, el mayor riesgo es que pasen desapercibidos durante el testing.

22

CASO DE ESTUDIO: FILA GENÉRICA.



23

CASO DE ESTUDIO: FILA GENÉRICA.

```
class FilaRobots extends FilaGenerica {
public FilaRobots(int n){
super(n);
}
public int robotsMenosEnergia(int e){
Robot r; int cont;
for (int i=0; i < cantElementos(); i++){
r = (Robot) f[i];
if (r.obtenerEnergia() < e)
cont++;
}
return cont;
}
}
```

24